v_rate_movies

Movie Ratings Bias Investigation, Cross-Platform Intelligence & Presentation Guide

Author: vartiwa (varunt154@gmail.com) • Repository: github.com/vartiwa/v_rate_movies

1. Executive Summary & Context

What is this project? *v_rate_movies* is a production-grade data-journalism replication, statistical audit, and interactive intelligence suite investigating movie rating bias on Fandango. It reproduces and extends Walt Hickey's landmark 2015 *FiveThirtyEight* investigation: *'Be Suspicious Of Online Movie Ratings, Especially Fandango's'*.

The Core Mystery: In 2015, Hickey noticed that movie star ratings displayed on Fandango were almost universally higher than ratings on Rotten Tomatoes, Metacritic, and IMDB. A closer inspection of Fandango's raw HTML revealed that the visual star icons presented to users did not match the underlying calculated ratings.

+0.24 ★	**89.0%**	**4.09 ★ vs 3.85 ★**	**3.89 ★**	**$p < 10^{\blacksquare 1 \blacksquare}$**
Avg Inflation Delta	Films Rounded Up	Displayed vs True HTML	2016–17 Post Drop	Paired t-test (d = 1.34)

2. The Mechanism: How the Rounding Glitch Worked

Fandango displayed star ratings in increments of half-stars (3.5, 4.0, 4.5, 5.0). Standard mathematical half-rounding rounds a value to the nearest 0.5 increment (e.g., 4.1 rounds to 4.0; 4.3 rounds to 4.5).

However, Fandango's frontend software contained an asymmetric, ceiling-biased algorithm:

- **The Glitch Rule:** If a film's true calculated score had any fractional part as small as **0.1**, Fandango's code bumped it up to the next half-star (e.g., **4.1** → **4.5★**; **4.6** → **5.0★**).
- **Ceiling Inflation:** 130 of 146 films (**89.04%**) had displayed stars strictly higher than their true HTML ratings. Zero films were rounded down.
- **Maximum Discrepancy:** Several major films received an unearned boost of **+0.50 full stars** (e.g., *Avengers: Age of Ultron*, *Cinderella*, *Ant-Man*, *The Water Diviner*).

True HTML Rating	Standard Math Round	Fandango Displayed	Artificial Delta	Effect Status
4.1 / 5.0	4.0 ★	**4.5 ★**	+0.4 ★	Glitch Boosted
4.2 / 5.0	4.0 ★	**4.5 ★**	+0.3 ★	Glitch Boosted
4.5 / 5.0	4.5 ★	**5.0 ★**	+0.5 ★	Glitch Boosted
4.6 / 5.0	4.5 ★	**5.0 ★**	+0.4 ★	Glitch Boosted
4.0 / 5.0	4.0 ★	4.0 ★	0.0 ★	Exact Match

3. Cross-Platform Benchmark & Critic Disparities

To benchmark Fandango against the broader movie ecosystem, all ratings were normalized onto a 0 to 5-star scale (Rotten Tomatoes / 20, Metacritic / 20, IMDB / 2):

Review Platform	Mean Rating	Median	Std Dev	Range	Distribution Character
Fandango (Displayed)	**4.09 ★**	4.00 ★	0.54	3.0 - 5.0	Severe Rightward Skew
Fandango (Actual HTML)	3.85 ★	3.90 ★	0.50	2.7 - 4.8	Moderate Right Skew
IMDB (Normalized)	3.37 ★	3.30 ★	0.48	2.0 - 4.3	Bell-shaped Normal
Rotten Tomatoes (Norm)	3.04 ★	3.00 ★	1.51	0.2 - 4.9	Wide Uniform Spread
Metacritic (Norm)	2.94 ★	2.95 ★	0.98	0.6 - 4.7	True Normal Gaussian

Egregious Critic Disparities: Films that were universally panned by critics were nonetheless awarded 4 to 5 stars on Fandango:

- *Do You Believe? (2015)*: Rotten Tomatoes gave it **18% (0.9★)**; Fandango displayed **5.0 Stars** (+4.10★ gap!).
- *Taken 3 (2015)*: Rotten Tomatoes gave it **9% (0.45★)**; Fandango displayed **4.5 Stars** (+4.05★ gap!).
- *Pixels (2015)*: Rotten Tomatoes gave it **17% (0.85★)**; Fandango displayed **4.5 Stars** (+3.65★ gap!).

4. The Business Conflict of Interest

Why did this happen? Understanding the commercial incentives explains why the glitch remained unaddressed for so long:

- **Aggregators vs. Brokers:** Rotten Tomatoes and Metacritic are media aggregators whose revenue is driven by pageviews and advertising. Fandango is an **online movie ticket broker** that collects convenience fees per ticket sold.
- **Conversion Optimization:** In e-commerce checkout funnels, higher ratings directly decrease user hesitation and increase ticket conversion rates. By ensuring no film ever displayed below 3.0 stars and 89% displayed 4.0+ stars, Fandango created an artificial halo effect that optimized ticket purchasing behavior.

5. Temporal Audit: Did Fandango Fix It? (2015 vs 2016–17)

In response to Hickey's report, Fandango claimed the issue was an accidental software bug and promised a remediation. An audit of 214 films released after the article (2016–2017) proves that Fandango indeed adjusted their display code:

- **Net Drop in Displayed Stars:** 2015 Displayed Mean = **4.089★** vs. 2016–17 Displayed Mean = **3.895★** (Drop of **-0.194★**).
- **Statistical Significance:** Welch's two-sample t-test: **t = 3.38 (p = 0.0008)**; Kolmogorov-Smirnov test: **D = 0.201 (p = 0.0003)**.
- **Alignment with Ground Truth:** The 2016–17 displayed distribution (3.89★) closely matched the 2015 true HTML baseline (3.85★), confirming that aggressive ceiling rounding had been eliminated.

6. Summary of Inferential Statistical Tests

Hypothesis Test	Null vs Alternative	Test Statistic	p-Value	Effect Size / CI	Verdict
Paired t-Test (2015 Inflation)	H■: μ_diff = 0 H■: μ_diff > 0	**t = 15.54**	**< 10■¹■**	d = 1.34 (Large)	**Reject H■**
Wilcoxon Signed-Rank	Non-parametric paired median test	**W = 0.00**	**< 10■¹■**	Rank biserial r = 1.0	**Reject H■**
Bootstrap 95% CI	10,000 resamples of mean inflation	Mean: +0.244	N/A	**[+0.213, +0.275] stars**	**Statistically Solid**
Welch's t-Test (Temporal Drop)	H■: μ_15 = μ_16-17 H■: μ_15 ≠ μ_16-17	**t = 3.38**	**0.0008**	d = 0.36 (Moderate)	**Reject H■**
Kolmogorov-Smirnov	Distribution shape equality test	**D = 0.201**	**0.0003**	Shift in CDF shape	**Reject H■**

7. Project Architecture & Codebase Structure

The project is structured with production modularity, automated test coverage, and clean separation of concerns:

Module / File	Responsibility & Key Capabilities
src/data_loader.py	Ingests and validates CSV datasets (2015 comparison, scrape, 2016-17), extracts film years safely, parses numeric columns.

src/analysis.py	Computes discrepancy frequencies, cross-platform stats, continuous 101-point Gaussian KDE curves, and temporal comparisons.
src/statistics.py	Inferential engine executing paired t-tests, Wilcoxon signed-rank, Welch's t-test, Kolmogorov-Smirnov, Cohen's d, and 10k bootstrap CI.
src/database.py	In-memory SQLite database manager supporting safe read-only SQL queries across fandango_2015, fandango_scrape, movie_ratings_16_17.
app/main.py	FastAPI ASGI backend serving JSON REST API endpoints (/api/overview, /api/movies, /api/sql, etc.) and web dashboard.
app/templates/index.html	Modern Bento Grid UI featuring glitch simulator, interactive parity plot, spotlight film card, and filter chips.
tests/ (24 pytest tests)	Comprehensive automated test suite validating schemas, ranges, math formulas, statistical p-values, and API endpoints.
vercel.json & api/index.py	Vercel serverless deployment configuration for 1-click global cloud deployment.

8. Project Presentation & Interview Pitch Guide

Use this cheat-sheet to explain the project with clarity and confidence during presentations, interviews, or reviews:

■ 30-Second Elevator Pitch:

'I built v_rate_movies, an end-to-end data analytics application investigating movie rating bias on Fandango. By comparing displayed star ratings with underlying HTML values across 146 theatrical releases, I proved that 89% of movies were artificially inflated due to an aggressive ceiling-rounding algorithm ($p < 10$■^{1}■). I packaged this analysis into a modern Bento Grid web application with interactive simulators, SQLite querying, and verified with 24 automated unit tests.'

■ 2-Minute Structured Walkthrough (5 Key Points):

1. **The Problem:** Fandango is an online ticket broker with an inherent conflict of interest. Higher ratings drive ticket conversions.
2. **The Discovery:** In 2015, Walt Hickey scraped raw HTML ratings and found they were consistently lower than the stars shown on screen.
3. **The Mathematics:** Fandango's frontend didn't use standard half-rounding; any fractional value ≥ 0.1 pushed a movie to the next half star (e.g. 4.1 → 4.5★).
4. **The Temporal Proof:** In 2016–17 follow-up data, displayed ratings dropped by ~0.20 stars ($p = 0.0008$), proving Fandango altered their algorithm after public exposure.
5. **The Engineering:** I engineered a clean Python analytics backend, automated statistical tests with SciPy/NumPy, and built a SaaS-grade Bento UI deployed to Vercel.

■ Anticipated Technical Q&A;:

Q: How did you prove the inflation was not random noise?
A: I conducted a paired Student's t-test on the differences (Displayed – HTML), yielding $t = 15.54$ ($p < 10$■^{1}■) and Cohen's $d = 1.34$. A 10,000-resample bootstrap confirmed a 95% confidence interval of [+0.213, +0.275] stars, entirely excluding zero.

Q: Why did you use Kernel Density Estimation (KDE) instead of simple histograms?
A: Discrete star ratings suffer from binning artifacts in histograms. Continuous Gaussian KDE with optimal bandwidth provides a smooth probability density function, clearly visualizing the rightward distribution shift and lack of sub-3.0 star ratings.

Q: How is the project deployed?
A: It is configured for 1-click serverless deployment on Vercel using an ASGI FastAPI bridge (api/index.py) and runs locally via run.bat or python -m app.main.

Project Links: GitHub: https://github.com/vartiwa/v_rate_movies • Author: vartiwa